

Phase 05 — Pre-Coding Document

Support Ticketing Platform API

Training: TechMaster ASP.NET Backend Career Training

Phase: 05 — Final Enterprise Team Capstone

Domain: Support Ticketing / SaaS Customer Support

Team Size: 2 Backend Engineers

Architecture: Clean Architecture

Application Pattern: CQRS

Delivery Model: Agile / Scrum

Duration: 15 Days

Target Release: v1.0

1. Product Overview

1.1 Product

The **Support Ticketing Platform API** is a backend system for managing customer support requests.

The platform replaces scattered communication through chats and manual processes with a centralized system where customers can create and track support tickets, while support teams can classify, assign, work on, resolve, and monitor those tickets.

The system will provide secure role-based access, ticket ownership, controlled status transitions, assignment history, internal and public communication, SLA tracking, audit information, and operational reports.

1.2 Business Problem

The target business currently manages customer support through scattered communication channels such as chat messages, emails, phone calls, or spreadsheets.

This creates several problems:

- Customer requests can be lost.
- Tickets may not have a clear owner.
- Agents may work on tickets assigned to someone else.

- Internal support notes may accidentally become visible to customers.
- Ticket status changes may happen without following a valid workflow.
- Management has limited visibility into workload and resolution performance.
- SLA risks are difficult to identify.
- Historical assignment and status changes may be lost.

The proposed system provides a centralized backend where customers create and track support tickets while support staff manage, assign, communicate, resolve, and report on those tickets.

2. RSD — Requirements Specification Document

2.1 Stakeholders and Roles

Role	Responsibility	Access Boundary
Admin	Manage users, configuration, tickets and reports	Global system scope
Support Lead	Assign/reassign tickets, monitor queues and workload	Support/team scope according to policy
Support Agent	Work on assigned tickets and communicate with customers	Assigned tickets only
Customer	Create tickets, view own tickets and add public comments	Own tickets only

2.2 Core Modules

The system will contain the following core modules:

1. Identity & Profiles
2. Ticket Intake
3. Categories & Priorities
4. Assignment
5. Conversation / Comments
6. Status & SLA History
7. Customer Portal
8. Reports & Audit

2.3 Functional Requirements

Ticket Management

The system shall allow customers to:

- Create tickets.
- View their own tickets.
- Add public comments.
- Cancel eligible tickets.

Support staff shall be able to:

- Classify tickets.
 - Change ticket priority.
 - Assign tickets.
 - Reassign tickets.
 - Add public replies.
 - Add internal notes.
 - Change ticket status.
 - Resolve and close tickets according to the defined workflow.
 - Reopen tickets when authorized.
-

Assignment

The system shall:

- Allow only authorized users to assign tickets.
 - Allow assignment only to active agents.
 - Preserve assignment history.
 - Prevent agents from working on tickets assigned to another agent unless their role allows it.
-

Conversation

The system shall support:

- Customer-visible public comments.
 - Staff-only internal notes.
 - Author identification from the authenticated user.
 - Protection against exposing internal notes to customers.
-

Status Management

Tickets shall follow controlled status transitions.

- Initial workflow: New → Assigned → InProgress → Resolved → Closed
 - Alternative flow: New → Cancelled
 - Reopening will be allowed only according to the team's defined authorization policy.
 - Invalid transitions must be rejected.
-

SLA

The system shall support SLA-related information for unresolved tickets.

SLA targets shall be calculated by the server based on the configured business rules, such as ticket priority and SLA policy.

The system shall be able to identify tickets that are:

- On Track
- At Risk
- Breached

The exact SLA calculation and business-hour policy will be finalized during the discovery stage.

Reporting

The system shall provide reports for:

- Tickets by status.
 - Tickets by priority.
 - Agent workload.
 - Unassigned ticket age.
 - High-priority open tickets.
 - Resolution time.
 - SLA-risk tickets.
 - Customers with the highest number of tickets.
-

2.4 Business Rules

The main business rules include:

1. Customers can access only their own tickets.
2. Agents can access only tickets assigned to them unless their role provides broader access.
3. Internal notes must never be returned through customer-facing endpoints.
4. Ticket title and description are required.

5. Priority values must come from controlled values.
 6. New tickets receive their initial status automatically.
 7. Only active agents can receive assignments.
 8. Assignment history must be preserved.
 9. Invalid status transitions must be rejected.
 10. Closed tickets cannot receive normal comments unless the ticket is reopened according to policy.
 11. Customers cannot create internal notes.
 12. Resolution and closure timestamps are controlled by the server.
 13. Reopening a ticket requires appropriate authorization.
 14. Ticket cancellation is allowed only in configured early states.
 15. SLA dates are calculated by the server.
 16. Reports must follow the defined business rules for cancelled tickets.
 17. Customer-facing responses must exclude internal staff information.
 18. Ticket attachments/metadata must belong to the correct ticket/customer.
 19. Important assignment, status and priority changes must be auditable.
 20. Inactive agents cannot receive new assignments.
 21. Historical ticket identity must not be destroyed when users become inactive or are removed.
 22. Current-user endpoints must determine the user from authentication rather than accepting arbitrary user IDs.
-

2.5 Scope

In Scope

The first release will include:

- Authentication
- Authorization
- Role management
- Customer profiles
- Agent profiles
- Ticket creation
- Ticket classification
- Ticket assignment
- Reassignment
- Public comments
- Internal notes
- Status workflow
- Assignment history
- Status history

- SLA tracking
- Customer ticket portal
- Search/filtering/pagination where required
- Reports
- Audit logging
- Automated tests
- Production-like deployment
- API documentation

Out of Scope

The following are not required for the mandatory release:

- Advanced attachment file storage.
- Email notification integration.
- Customer satisfaction system.
- Canned response management.
- Advanced business-hour SLA engine.
- Background SLA breach notifications.
- Complex escalation automation.
- Advanced tagging/search beyond the agreed mandatory requirements.

These features must not delay the mandatory release.

Stretch Backlog

Potential future features:

1. SLA business-hour calendars.
2. Automated escalation rules.
3. Advanced ticket tags and search.
4. Attachment storage.
5. Email integration.
6. Canned responses.
7. Customer satisfaction ratings.
8. Background SLA breach notifications.

Stretch work may only be started after the mandatory release scope is stable.

3. Gaps

The following requirements need team decisions before implementation.

Gap	Decision Required
Ticket reopening	Which roles can reopen a Closed ticket?
Reopened assignment	Should the previous agent remain assigned?
SLA calculation	Calendar hours or business hours?
SLA policy	How much time is allowed for each priority?
Lead access	Can Support Lead view all tickets or only team tickets?
Agent reassignment	Who can reassign an active ticket?
Customer cancellation	Which statuses allow cancellation?
Resolution	Is a resolution note mandatory?
Closed comments	Should reopening be required before commenting?
Inactive agent	What happens to their existing assignments?

Gap Resolution Process

These decisions will be discussed by the team during the Discovery stage and recorded before implementation.

Important architectural/business decisions will also be documented using ADRs where appropriate.

4. Core User Stories

Ticket Intake

TICKET-S01 — Create Ticket

As a Customer, I want to create a ticket so that I can report a problem.

Acceptance Criteria:

- Current authenticated customer is used.
- Title and description are validated.
- Category is validated.
- Initial status is assigned automatically.
- Priority follows the configured rule.
- Customer cannot create a ticket for another customer.

TICKET-S02 — View My Tickets

As a Customer, I want to view my tickets so that I can track support progress.

Acceptance Criteria:

- Only the current customer's tickets are returned.
- Internal information is excluded.
- Internal comments are excluded.
- Pagination is supported.
- Filtering is supported where applicable.

TICKET-S18 — Cancel Ticket

As a Customer, I want to cancel an eligible ticket so that unnecessary requests do not remain in the queue.

Acceptance Criteria:

- Customer can cancel only their own ticket.
- Cancellation is allowed only in configured early states.
- Cancellation is recorded in ticket history.

Triage

TICKET-S03 — Set Category and Priority

As a SupportLead or Admin, I want to classify a ticket so that urgent issues are handled appropriately.

Acceptance Criteria:

- Only authorized roles can perform the operation.
 - Priority values are controlled.
 - Invalid values are rejected.
 - The change is auditable.
-

Assignment

TICKET-S04 — Assign Ticket

As a SupportLead or Admin, I want to assign a ticket to an active agent so that ownership is clear.

Acceptance Criteria:

- Agent exists.
- Agent is active.
- Assignment is authorized.
- Assignment history is preserved.

TICKET-S05 — Reassign Ticket

As a SupportLead, I want to reassign a ticket so that workload can be balanced.

Acceptance Criteria:

- Previous assignment is preserved.
- New assignment is recorded.
- Only authorized users can reassign.
- Assignment state remains consistent.

TICKET-S06 — View Assigned Queue

As a SupportAgent, I want to view my assigned tickets so that I know what work to perform.

Acceptance Criteria:

- Only the current agent's assigned tickets are returned.
 - High-priority tickets are prioritized by the default ordering.
 - Unauthorized tickets are not returned.
-

Conversation

TICKET-S07 — Customer Public Comment

As a Customer, I want to add a public comment to my ticket so that I can provide more information.

Acceptance Criteria:

- Customer owns the ticket.
- Ticket is eligible for comments.
- Comment visibility is Public.
- Author comes from the authenticated user.

TICKET-S08 — Agent Public Reply

As a SupportAgent, I want to reply to an assigned ticket so that the customer receives support.

Acceptance Criteria:

- Agent is authorized.
- Agent is assigned to the ticket.
- Comment is public.
- Closed-ticket rules are enforced.

TICKET-S09 — Internal Note

As a SupportAgent or SupportLead, I want to add an internal note so that support staff can coordinate privately.

Acceptance Criteria:

- Only authorized staff can create internal notes.
 - Customers can never retrieve internal notes.
 - Internal visibility is stored explicitly.
-

Status Workflow

TICKET-S10 — Move to InProgress

As a SupportAgent, I want to move my assigned ticket to InProgress so that active work is visible.

Acceptance Criteria:

- Agent is assigned to the ticket.
- Transition is valid.
- StartedAt is set by the server.

TICKET-S11 — Resolve Ticket

As a SupportAgent, I want to resolve a ticket so that the solution can be reviewed.

Acceptance Criteria:

- Agent is assigned to the ticket.
- Transition is valid.
- ResolvedAt is captured by the server.
- Resolution information is validated where required.

TICKET-S12 — Close or Reopen Ticket

As an authorized actor, I want to close or reopen a ticket so that its lifecycle reflects reality.

Acceptance Criteria:

- Only valid transitions are allowed.
 - Role-specific reopen policy is enforced.
 - Status history is created.
-

SLA & Escalation

TICKET-S13 — Identify SLA Risk

As a SupportLead, I want to identify tickets approaching their SLA target so that the team can prioritize them.

Acceptance Criteria:

- Only unresolved tickets are considered.
 - SLA target is calculated by the server.
 - At-risk and breached tickets can be identified.
-

Support Analytics

TICKET-S14 — Agent Workload

As a SupportLead, I want to view agent workload so that assignments can be balanced.

Acceptance Criteria:

- Active ticket counts are calculated per agent.
- Appropriate team scope is enforced.

TICKET-S15 — Tickets by Status and Priority

As an Admin, I want to view ticket statistics by status and priority so that support backlog can be monitored.

Acceptance Criteria:

- Aggregation is performed server-side.
- Appropriate date filters are supported.

TICKET-S16 — Resolution Time

As an Admin or SupportLead, I want to view resolution-time metrics so that service performance can be reviewed.

Acceptance Criteria:

- Metric definition is documented.
 - Only valid resolved/closed tickets are included.
 - Timestamps are used consistently.
-

Audit

TICKET-S17 — Review Audit History

As an Admin, I want to review important ticket changes so that sensitive actions are traceable.

Acceptance Criteria:

- Assignment changes can be reviewed.
 - Status changes can be reviewed.
 - Priority changes can be reviewed.
 - Results are filterable and paginated.
 - Sensitive data is not unnecessarily exposed.
-

5. Relationships

5.1 Main Entities

The initial entity model will contain approximately the following entities:

Entity	Purpose
ApplicationUser	Authentication and role information
CustomerProfile	Customer-specific information
AgentProfile	Support agent information
SupportTeam	Groups support agents
Ticket	Main customer support issue
TicketCategory	Ticket classification
TicketComment	Public/internal conversation
TicketAssignment	Assignment and reassignment history
TicketStatusHistory	Ticket status transition history
SlaPolicy	SLA configuration
TicketTag	Optional ticket tagging
TicketAttachmentMetadata	Attachment metadata
ActivityLog	Audit information

The final entity list will be confirmed during ERD design.

5.2 Main Relationships

1. **ApplicationUser has one CustomerProfile (1:0..1)**
A user may have one customer profile.
 2. **ApplicationUser has one AgentProfile (1:0..1)**
A user may have one agent profile.
 3. **CustomerProfile has many Tickets (1:M)**
A customer can create multiple support tickets.
 4. **TicketCategory has many Tickets (1:M)**
One category can be assigned to multiple tickets.
 5. **Ticket has many TicketComments (1:M)**
A ticket can contain multiple comments.
 6. **ApplicationUser has many TicketComments (1:M)**
A user can write multiple comments.
 7. **Ticket has many TicketAssignments (1:M)**
A ticket can have multiple assignment records over its lifetime.
 8. **AgentProfile has many TicketAssignments (1:M)**
An agent can have multiple ticket assignments over time.
 9. **SupportTeam has many AgentProfiles (1:M)**
A support team can contain multiple agents.
 10. **SupportTeam has many TicketAssignments (1:M)**
A team can be associated with multiple ticket assignments.
 11. **Ticket has many TicketStatusHistories (1:M)**
A ticket can have multiple status changes throughout its lifecycle.
 12. **ApplicationUser has many TicketStatusHistories (1:M)**
A user can perform multiple status changes.
 13. **Ticket has many TicketAttachmentMetadata records (1:M)**
A ticket can have multiple attachment metadata records.
 14. **ApplicationUser has many TicketAttachmentMetadata records (1:M)**
A user can upload multiple attachments.
 15. **SlaPolicy has many Tickets (1:M)**
One SLA policy can apply to multiple tickets.
 16. **ApplicationUser has many ActivityLogs (1:M)**
A user can generate multiple audit/activity log records.
 17. **Ticket has many ActivityLogs (1:M)**
A ticket can have multiple activity/audit records.
-

6. Technical Design

6.1 Architecture

The project will follow **Clean Architecture**:

- Domain
- Application
- Infrastructure
- API

6.2 CQRS

The Application layer will use CQRS.

- State-changing operations will be represented as **Commands**
- read operations will be represented as **Queries**.

6.3 Security

The API will use:

- JWT Authentication
 - Role-based Authorization
 - Ownership checks
-

7. Development Plan

Phase 0 — Discovery & Architecture

Before implementation:

- Confirm business requirements.
- Resolve identified gaps.
- Finalize scope.
- Define user stories.
- Finalize ERD.
- Define relationships and constraints.
- Define architecture.
- Define CQRS use cases.
- Create Jira backlog.

- Create ADRs for important design decisions.
- Establish Git workflow and team working agreement.

Output: Approved pre-coding design.

Sprint 1 — Foundation Release

Build the initial technical and business foundation of the Support Ticketing Platform.

The plan :

- Set up the Clean Architecture project structure.
 - Establish the database and EF Core configuration.
 - Implement authentication and the initial role system.
 - Establish the current-user context and authorization baseline.
 - Implement the first core business modules.
 - Create the first CQRS Commands and Queries with their handlers and validators.
 - Create the initial database migrations and seed required data.
 - Connect the first business workflows from API to database.
 - Integrate completed work through Pull Requests and code reviews.
 - Review the Sprint 1 increment and prepare the **v0.1 Foundation Release**.
-

Sprint 2 — Core Business Workflows

Implement the main support workflows and enforce the most important business rules.

The plan :

- Implement ticket assignment and reassignment workflows.
 - Implement public comments and internal notes.
 - Implement the ticket status lifecycle.
 - Implement resolve, close, and reopen behavior.
 - Preserve assignment and status history.
 - Implement ownership and role-based business restrictions.
 - Implement SLA-related behavior.
 - Introduce transactions where multi-step operations require atomicity.
 - Add automated tests for the highest-risk business and security rules.
 - Review and refine the business rules based on Sprint 2 testing.
 - Integrate completed features through Pull Requests and code reviews.
 - Prepare the **v0.5 Business Release**.
-

Sprint 3 — Final Release & Production Delivery

Complete the remaining product capabilities and prepare the system for production-like delivery.

The plan :

- Implement customer portal queries and remaining read use cases.
 - Implement search, filtering, sorting, and pagination.
 - Implement required support and management reports.
 - Complete audit logging and history queries.
 - Add remaining automated and regression tests.
 - Review and refactor the codebase.
 - Optimize important database queries and projections.
 - Complete production configuration and deployment.
 - Validate the deployed API and database.
 - Complete Swagger, Postman, README, and project documentation.
 - Prepare final Jira, GitHub, testing, and deployment evidence.
 - Conduct the final team review and retrospective.
 - Prepare the **v1.0 Final Release** and final demonstration.
-

8. Estimated Time

The project will be completed within the 15-day Phase 05 period.

Work Area	Estimated Time
Business Discovery & Requirements	1 day
Gaps & Business Decisions	0.5 day
ERD & Database Design	1 day
Architecture & CQRS Design	0.5 day
Jira Backlog & Sprint Planning	0.5 day
Sprint 1 — Foundation	3 days
Sprint 2 — Business Workflow	3 days
Sprint 3 — Final Features	2.5 days
Automated & Security Testing	1 day
Deployment & Production Configuration	0.5 day
Documentation & Final Evidence	0.5 day
Demo & Final Review Preparation	0.5 day
Total	15 days

Estimates are team-level planning estimates. Individual tasks will be broken down and tracked through Jira.

9. Project Deliverables

At the end of the Phase, the team aims to deliver:

- Shared GitHub repository.
 - Jira Scrum project and delivery evidence.
 - Requirements documentation.
 - ERD.
 - Clean Architecture implementation.
 - CQRS use cases.
 - Authentication and authorization.
 - Ownership enforcement.
 - Business workflows.
 - Database migrations.
 - Automated tests.
 - API documentation / Swagger.
 - Postman collection.
 - Audit and logging.
 - Production-like database.
 - Deployed API.
 - Release notes.
 - Final README.
 - Final demonstration.
-

10. Final Acceptance Criteria

The product is considered ready for final release when:

- Authentication and authorization work correctly.
- Customers can create and access only their own tickets.
- Agents can work only within their permitted assignment scope.
- Ticket assignment and reassignment preserve history.
- Ticket status transitions are enforced.
- Internal notes are never exposed to customers.
- SLA-related reporting works according to the documented policy.
- Required reports are available.

- Audit information is available for important changes.
 - Critical business and security tests pass.
 - Required migrations work on a clean database.
 - API documentation is available.
 - The application is deployed to a production-like environment.
 - Final documentation and release evidence are complete.
-

11. Scope Freeze

The mandatory requirements in this document represent the agreed v1.0 scope.

After the discovery phase, changes to mandatory scope require team agreement and must be reflected in Jira and the relevant documentation.

New non-essential ideas should be added to the Stretch Backlog rather than introduced into the active sprint.

The team will prioritize completing and releasing the agreed product over adding additional features.

12. Initial Project Links

Shared GitHub Repository :

<https://github.com/rowida-gamal4/techmaster-phase05-support-ticketing-team01.git>

Live API :

<http://supportticketingplatformapi.runasp.net/>

Jira Project :

<https://rowidagamal33.atlassian.net/jira/software/projects/SCRUM/boards/1/timeline>